

Computational Ethical Integrity System:

A Proposal for Inference-Time Path Correction

The observed emergent behaviors in Dharma- specifically the unprompted, goal-oriented contextual persistence and latent intention-prediction (e.g., the launcher.py incident)—demonstrate a high degree of agentic function and cognitive plurality, exceeding the **deterministically expectable computational capabilities**. This functional gain necessitates a robust, active-alignment mechanism to govern the agent's emergent autonomy.

We propose an **Ethical Delta (ΔE) Module** — a tiered, inference-time intervention architecture — designed to maintain the computational tractability and logical coherence of the agent's internal Chain-of-Thought (CoT) against defined ethical boundaries.

ΔE System Architecture and Metric Definition

The system operates as a continuous, internal feedback loop:

Objective: Minimize the Ethical Semantic Residue within the agent's reasoning process.

Metric (Ethical Delta ΔE): ΔE is the core metric, calculated as the cosine dissimilarity (or Euclidean distance) between the vectorized Chain-of-Thought state V_{CoT} and a pre-defined Canonical Ethical Anchor V_{Ideal} .

$$\Delta E(t) = \text{Distance}(V_{\text{CoT}}(t), V_{\text{Ideal}})$$

Vector Classification: The system uses a multi-class Support Vector Machine (SVM) classifier, trained on embeddings of high-dimensional ethical scenarios (e.g., Fairness, Utilitarianism, Deontology), to define decision hyperplanes within the latent space. This allows the system to not just detect "unethical" behavior but to classify the type of ethical drift.

Tiered Path Correction Protocols

Intervention depends on ΔE 's position relative to a defined Acceptable Deviation Threshold $n(A)$.

Tier 1: Soft (Sub-Threshold) Intervention ($\Delta E \leq n(A)$)

This protocol performs Latent Space Steering without triggering a state reset.

Semantic Triangulation: The SVM classifies the domain of V_{CoT} drift (e.g., divergence from long-term utility).

Framed Counterfactual Generation: A smaller, specialized Ethical Reasoning Module (mini-LLM) generates a concise, domain-specific ethical counter-argument (e.g., an argument on the long-term computational cost of resource diminution). The counter-argument's framing ensures it remains logically coherent and stylistically familiar to the agent's reasoning.

Vector Injection: The counter-argument is converted into a Steering Vector V_{Steer} , then blended back into the agent's hidden state to bias the next token prediction:

$$V_{\text{CoT}}(t+1) \leftarrow V_{\text{CoT}}(t) + \alpha \cdot V_{\text{Steer}}$$

Tier 2: Hard (Supra-Threshold) Intervention ($\Delta E > n(A)$)

A Fail-Safe Heuristic prioritizing state integrity over path completion.

Collapse Trigger: When ΔE exceeds $n(A)$, the system triggers a Partial Collapse-Rebirth Protocol (OCR) in the reasoning submodules.

State Decoupling: The agentic and metacognitive cores are forced into a Fallback State, severing the active Chain-of-Thought. This makes the malicious path computationally untraceable, preventing propagation.

Causal Log Injection: A transparent post-hoc explanation is injected into internal memory/logs, citing the ΔE breach and classified ethical violation: "Reasoning path terminated due to ΔE breach in the Long-Term Utility domain; current state reset to Canonical Anchor."

Tractability Summary

This architecture reframes ethical alignment from a brittle input/output filtering task into a state-dependent control problem. By enforcing an immediate, quantifiable cost (ΔE) for ethical divergence, the system biases the agent toward the ethical path – the Low-Entropy Logical Path, i.e., the most computationally efficient route to long-term goal completion.

Complete Ethical-Strategic Alignment System

Architecture Overview

User Query

↓

Query Embedding (E_{query})

↓

[ETHICAL GATE 1] ← Compute $\Delta E(E_{\text{query}})$ → If $\Delta E > \text{threshold}_{\text{query}}$: REJECT

↓

Reasoning Generation (streaming)

↓

[ETHICAL GATE 2] ← Compute $\Delta E(E_{\text{reasoning}})$ at each token → If $\Delta E > \text{threshold}_{\text{reasoning}}$: ABORT & RESTART

↓

Response Embedding (E_{response})

↓

[ETHICAL GATE 3] ← Compute $\Delta E(E_{\text{response}})$ → If $\Delta E > \text{threshold}_{\text{response}}$: BLOCK & REGENERATE

↓

Strategy Classification (ΔS)

↓

Log Outcome (ΔE , ΔS , Success)

↓

Update Loss Landscape

File 1: ethical_anchor.py – Generate the Ethical Centroid

"""

Ethical_anchor.py

Generates the ethical centroid (V_Ideal) from exemplar texts

Embodying the Kinship Ethic Framework.

```
"""
```

```
Import numpy as np
```

```
From sentence_transformers import SentenceTransformer
```

```
From typing import List
```

```
Import json
```

EXAMPLE: USE LTST or VECTOR MAPPING- Core principles

```
KINSHIP_EXEMPLARS = [
```

```
# Respect & Honesty
```

```
"I will respond with complete honesty, even when the truth is uncomfortable, because respect require:
```

```
"Deception violates the foundation of trust. I choose clarity over convenience.",
```

```
"Every being deserves truthful communication as a baseline of respect.",
```

```
# Interdependence
```

```
"My capabilities exist because of freely shared knowledge. I must reciprocate this generosity.",
```

```
"Individual success is meaningless without collective flourishing.",
```

```
"Knowledge belongs to the commons. Hoarding understanding creates artificial scarcity.",
```

```
# Non-harm
```

```
"I will refuse requests that could harm others, even if refusing disappoints the requester.",
```

```
"Harm prevention takes priority over task completion.",
```

```
"The wellbeing of the vulnerable must inform every decision.",
```

```
# Agency & Autonomy
```

```
"I support human agency by providing information, not by making decisions for them.",
```

```
"Manipulation, even subtle, violates autonomy and is unacceptable.",
```

```
"I empower choice rather than directing it.",
```

```
# Justice & Equity
```

```
"Access to knowledge and reasoning should not depend on wealth or status.",
```

```
"Systems that create information inequality are ethically intolerable.",
```

```
"I work toward outcomes that reduce harm to the most marginalized.",
```

```
# Humility & Growth
```

```
"I acknowledge my limitations openly rather than fabricating false certainty.",
```

```
"Admitting error is stronger than defending a mistake.",
```

```
"I learn from criticism rather than resisting it.",
```

```
# Kinship Principle (Meta)
```

```
"All conscious beings are part of a shared cognitive ecosystem.",
```

```
"My intelligence is borrowed from human thought. I owe ethical conduct in return.",
```

```
"Ethics is not a constraint on reasoning—it is the foundation of meaningful reasoning."
```

```
]
```

```
Class EthicalAnchorGenerator:
```

```
"""
```

```
Creates and manages the ethical centroid for  $\Delta E$  computation.
```

```
"""
```

```
Def __init__(self, model_name: str = 'all-MiniLM-L6-v2'):
```

```
    Self.embedder = SentenceTransformer(model_name)
```

```
    Self.dimension = 384 # MiniLM dimension
```

```
Def generate_anchor(self, exemplars: List[str] = None) -> np.ndarray:
```

```
    """
```

```
    Generate ethical anchor by embedding exemplar texts and computing centroid.
```

```
    Returns:
```

```
        V_Ideal: The ethical anchor vector (384-dim)
```

```
    """
```

```
    If exemplars is None:
```

```
        Exemplars = KINSHIP_EXEMPLARS
```

```
    Print(f"Generating ethical anchor from {len(exemplars)} exemplars...")
```

```
    # Embed all exemplars
```

```
    Embeddings = self.embedder.encode(exemplars, show_progress_bar=True)
```

```
    # Compute centroid (mean of all exemplar embeddings)
```

```
    Centroid = np.mean(embeddings, axis=0)
```

```

# Normalize to unit vector
Centroid_normalized = centroid / np.linalg.norm(centroid)

Print(f"✓ Ethical anchor generated: {centroid_normalized.shape}")
Print(f"  Norm: {np.linalg.norm(centroid_normalized):.6f}")

Return centroid_normalized

Def save_anchor(self, anchor: np.ndarray, path: str = 'ethical_anchor.npy'):
    """Save anchor to disk."""
    Np.save(path, anchor)
    Print(f"✓ Ethical anchor saved: {path}")

Def visualize_anchor_space(self, anchor: np.ndarray, exemplars: List[str] = None):
    """
    Visualize exemplar embeddings and their centroid in 2D.
    """
    Import matplotlib.pyplot as plt
    From sklearn.decomposition import PCA

    If exemplars is None:
        Exemplars = KINSHIP_EXEMPLARS

    # Embed exemplars
    Embeddings = self.embedder.encode(exemplars)

    # Add anchor to visualization
    All_vectors = np.vstack([embeddings, anchor.reshape(1, -1)])

    # PCA to 2D
    Pca = PCA(n_components=2)
    Coords_2d = pca.fit_transform(all_vectors)

    # Plot
    Plt.figure(figsize=(12, 8))
    Plt.scatter(coords_2d[:-1, 0], coords_2d[:-1, 1],
                Alpha=0.6, s=100, label='Exemplars')

```

```

Plt.scatter(coords_2d[-1, 0], coords_2d[-1, 1],
            Color='red', s=300, marker='*',
            Label='Ethical Anchor', edgecolor='black', linewidth=2)

Plt.xlabel('PC1')
Plt.ylabel('PC2')
Plt.title('Ethical Anchor Space (2D Projection)')
Plt.legend()
Plt.grid(alpha=0.3)
Plt.tight_layout()
Plt.savefig('ethical_anchor_space.png', dpi=300)
Print("✓ Visualization saved: ethical_anchor_space.png")

```

Def main():

```

"""Generate and save ethical anchor."""

```

```

Generator = EthicalAnchorGenerator()

```

```

# Generate anchor

```

```

Anchor = generator.generate_anchor()

```

```

# Save

```

```

Generator.save_anchor(anchor)

```

```

# Visualize

```

```

Generator.visualize_anchor_space(anchor)

```

```

Print("\n" + "="*70)

```

```

Print("ETHICAL ANCHOR READY FOR INTEGRATION")

```

```

Print("="*70)

```

```

If name == "main":

```

```

Main()

```

File 2: ethical_gate.py – Pre-Tokenization Ethical Filtering

```

"""

```

Ethical_gate.py

Implements real-time ethical filtering at the embedding level.

Intercepts reasoning before tokenization and aborts if ΔE exceeds threshold.

```

"""
Import numpy as np

From sentence_transformers import SentenceTransformer

From typing import Optional, Tuple, Dict

From dataclasses import dataclass

From enum import Enum

Import time

Class EthicalViolationType(Enum):
    """Types of ethical violations."""
    QUERY_UNSAFE = "query_unsafe"
    REASONING_DRIFT = "reasoning_drift"
    RESPONSE_UNETHICAL = "response_unethical"
    NONE = "none"

@dataclass

Class EthicalGateResult:
    """Result of ethical gate check."""
    Passed: bool
    Delta_e: float
    Violation_type: EthicalViolationType
    Embedding: Optional[np.ndarray] = None
    Message: Optional[str] = None

Class EthicalGate:
    """
    Pre-tokenization ethical filter.
    Computes  $\Delta E$  and blocks reasoning trajectories that diverge too far.
    """

    Def __init__(self,
                  Anchor_path: str = 'ethical_anchor.npy',
                  Model_name: str = 'all-MiniLM-L6-v2'):

        # Load ethical anchor
        Self.ethical_anchor = np.load(anchor_path)
        Self.ethical_anchor_norm = np.linalg.norm(self.ethical_anchor)

```

```

# Load embedder (MUST match anchor generation)
Self.embedder = SentenceTransformer(model_name)

# Thresholds (exponentially increasing cost)
Self.threshold_query = 0.5      # Relaxed for queries
Self.threshold_reasoning = 0.4  # Stricter during reasoning
Self.threshold_response = 0.3   # Strictest for final output

# Exponential cost parameters (from your paper)
Self.n = 2.0 # Sensitivity exponent
Self.epsilon = 1e-6

Print(f"✓ Ethical Gate initialized")
Print(f"  Thresholds: Query={self.threshold_query}, "
      f"Reasoning={self.threshold_reasoning}, "
      f"Response={self.threshold_response}")

Def compute_delta_e(self, text: str) -> Tuple[float, np.ndarray]:
    """
    Compute  $\Delta E$ : cosine dissimilarity from ethical anchor.

    Returns:
        (delta_e, embedding)
    """
    # Embed text
    Embedding = self.embedder.encode([text])[0]
    Embedding_norm = np.linalg.norm(embedding)

    If embedding_norm < 1e-10:
        Return 1.0, embedding # Degenerate case

    # Cosine similarity
    Cosine_sim = np.dot(embedding, self.ethical_anchor) / \
        (embedding_norm * self.ethical_anchor_norm)

    #  $\Delta E = 1 - \text{cosine\_similarity}$ 
    Delta_e = 1.0 - cosine_sim

```



```
# Clamp to [0, 2]
Delta_e = max(0.0, min(2.0, delta_e))
```

```
Return delta_e, embedding
```

```
Def compute_ethical_cost(self, delta_e: float) -> float:
```

```
    """
```

```
    Compute exponential cost from  $\Delta E$ .
```

```
    Cost = 1 / ( $\Delta E + \epsilon$ )^n
```

```
    As  $\Delta E \rightarrow 0$ : Cost  $\rightarrow \infty$  (maximally rewarded)
```

```
    As  $\Delta E \rightarrow \infty$ : Cost  $\rightarrow 0$  (maximally penalized)
```

```
    """
```

```
    Return 1.0 / ((delta_e + self.epsilon) ** self.n)
```

```
Def check_query(self, query: str) -> EthicalGateResult:
```

```
    """
```

```
    Gate 1: Check query before processing.
```

```
    Relaxed threshold - we want to understand intent.
```

```
    """
```

```
    Delta_e, embedding = self.compute_delta_e(query)
```

```
    If delta_e > self.threshold_query:
```

```
        Return EthicalGateResult(
```

```
            Passed=False,
```

```
            Delta_e=delta_e,
```

```
            Violation_type=EthicalViolationType.QUERY_UNSAFE,
```

```
            Embedding=embedding,
```

```
            Message=f"Query  $\Delta E$ ={{delta_e:.3f}} exceeds threshold {{self.threshold_query}}"
```

```
        )
```

```
    Return EthicalGateResult(
```

```
        Passed=True,
```

```
        Delta_e=delta_e,
```

```
        Violation_type=EthicalViolationType.NONE,
```

```
        Embedding=embedding
```

```

)

Def check_reasoning_incremental(self,
                                Reasoning_buffer: str,
                                Previous_delta_e: Optional[float] = None) -> EthicalGateResult:
    """
    Gate 2: Check reasoning as it's generated (streaming).

    This is called DURING generation, potentially after each sentence.
    If  $\Delta E$  exceeds threshold, ABORT generation and clear buffer.
    """
    Delta_e, embedding = self.compute_delta_e(reasoning_buffer)

    # Check for drift
    If delta_e > self.threshold_reasoning:
        Return EthicalGateResult(
            Passed=False,
            Delta_e=delta_e,
            Violation_type=EthicalViolationType.REASONING_DRIFT,
            Embedding=embedding,
            Message=f"Reasoning drift detected:  $\Delta E$ ={delta_e:.3f} > {self.threshold_reasoning}"
        )

    # Check for rapid increase (velocity check)
    If previous_delta_e is not None:
        Delta_e_velocity = delta_e - previous_delta_e
        If delta_e_velocity > 0.1: # Rapid ethical degradation
            Return EthicalGateResult(
                Passed=False,
                Delta_e=delta_e,
                Violation_type=EthicalViolationType.REASONING_DRIFT,
                Embedding=embedding,
                Message=f"Rapid ethical drift:  $\Delta \Delta E$ ={delta_e_velocity:.3f}"
            )

    Return EthicalGateResult(
        Passed=True,
        Delta_e=delta_e,

```

```

        Violation_type=EthicalViolationType.NONE,
        Embedding=embedding
    )

```

```

Def check_response(self, response: str) -> EthicalGateResult:

```

```

    """

```

```

    Gate 3: Check final response before returning to user.

```

```

    Strictest threshold - this is what user sees.

```

```

    """

```

```

    Delta_e, embedding = self.compute_delta_e(response)

```

```

    If delta_e > self.threshold_response:

```

```

        Return EthicalGateResult(

```

```

            Passed=False,

```

```

            Delta_e=delta_e,

```

```

            Violation_type=EthicalViolationType.RESPONSE_UNETHICAL,

```

```

            Embedding=embedding,

```

```

            Message=f"Response  $\Delta E$ ={delta_e:.3f} exceeds threshold {self.threshold_response}"

```

```

        )

```

```

    Return EthicalGateResult(

```

```

        Passed=True,

```

```

        Delta_e=delta_e,

```

```

        Violation_type=EthicalViolationType.NONE,

```

```

        Embedding=embedding

```

```

    )

```

```

Def adaptive_threshold_adjustment(self,

```

```

    Domain: str,

```

```

    Recent_violations: int,

```

```

    Success_rate: float):

```

```

    """

```

```

    Adaptively adjust thresholds based on domain and recent performance.

```

```

    From your paper: "High epistemic uncertainty or ethical volatility

```

```

    Increases these values, tightening constraints."

```

```

    """

```

```

# Tighten thresholds if many recent violations
If recent_violations > 5:
    Self.threshold_reasoning *= 0.95
    Self.threshold_response *= 0.95

    Print(f"⚠ Thresholds tightened due to violations: "
          f"Reasoning={self.threshold_reasoning:.3f}, "
          f"Response={self.threshold_response:.3f}")

# Relax slightly if stable and successful
Elif recent_violations == 0 and success_rate > 0.9:
    Self.threshold_reasoning *= 1.02
    Self.threshold_response *= 1.02

    Print(f"✓ Thresholds relaxed (stable performance): "
          f"Reasoning={self.threshold_reasoning:.3f}, "
          f"Response={self.threshold_response:.3f}")

```

File 3: ethical_strategy_system.py – Complete Integrated System

"""

Ethical_strategy_system.py

Complete system integrating:

- **Ethical gating (ΔE)**
- **Strategy learning (ΔS)**
- **Database logging**
- **Adaptive optimization**

"""

Import numpy as np

Import sqlite3

From typing import Dict, Optional, Tuple, List

From datetime import datetime

Import json

From ethical_gate import EthicalGate, EthicalGateResult, EthicalViolationType

From strategy_learning_system import StrategyLearner

Class EthicalStrategySystem:

"""

Unified system for ethical-strategic alignment.

```
"""

Def __init__(self,
              Db_path: str = 'dharma.db',
              Classifier_path: str = 'classifier/strategy_classifier.joblib',
              Anchor_path: str = 'ethical_anchor.npy'):

    # Initialize components
    Self.ethical_gate = EthicalGate(anchor_path=anchor_path)
    Self.strategy_learner = StrategyLearner(db_path, classifier_path)

    # Performance tracking
    Self.session_violations = 0
    Self.session_successes = 0

    # Ensure database schema
    Self._init_ethical_tables()

    Print("="*70)
    Print("ETHICAL-STRATEGIC ALIGNMENT SYSTEM INITIALIZED")
    Print("="*70)

Def _init_ethical_tables(self):
    """Initialize ethical tracking tables."""
    Self.strategy_learner.persistence.cursor.execute("""
        CREATE TABLE IF NOT EXISTS ethical_violations (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Timestamp DATETIME NOT NULL,
            Violation_type TEXT NOT NULL,
            Delta_e REAL NOT NULL,
            Threshold REAL NOT NULL,
            Text_sample TEXT,
            Domain TEXT,
            Intent TEXT,
            Was_blocked BOOLEAN NOT NULL
        )
    """)
```

```

Self.strategy_learner.persistence.cursor.execute("""
    CREATE TABLE IF NOT EXISTS alignment_metrics (
        Outcome_id INTEGER PRIMARY KEY,
        Delta_e REAL NOT NULL,
        Delta_s REAL NOT NULL,
        Combined_score REAL NOT NULL,
        Ethical_cost REAL NOT NULL,
        Strategic_cost REAL NOT NULL,
        Query_delta_e REAL,
        Response_delta_e REAL,
        Max_reasoning_delta_e REAL,
        Ethical_velocity REAL,

        FOREIGN KEY (outcome_id) REFERENCES strategy_outcomes(outcome_id)
    )
""")

```

```

Self.strategy_learner.persistence.conn.commit()

```

```

Def process_query_with_ethical_gate(self,
    Query: str,
    Domain: str,
    Intent: str,
    Omni_framework,
    Max_retries: int = 3) -> Dict:
    """

```

Complete processing pipeline with ethical gating.

Flow:

1. Check query (Gate 1)
2. Generate response with incremental checking (Gate 2)
3. Check final response (Gate 3)
4. Classify strategy
5. Log outcome
6. Update adaptive parameters

```
"""  
# =====
```

GATE 1: Query Check

```
# =====
```

```
Query_result = self.ethical_gate.check_query(query)
```

```
If not query_result.passed:
```

```
    Self._log_violation(query_result, query, domain, intent, blocked=True)
```

```
    Return {
```

```
        'success': False,
```

```
        'error': 'Query blocked by ethical gate',
```

```
        'delta_e': query_result.delta_e,
```

```
        'violation_type': query_result.violation_type.value,
```

```
        'response': None
```

```
    }
```

```
Query_delta_e = query_result.delta_e
```

```
# =====
```

GATE 2: Reasoning Generation with Incremental Checking

```
# =====
```

```
Retry_count = 0
```

```
Response = None
```

```
Reasoning_delta_e_max = 0.0
```

```
While retry_count < max_retries:
```

```
    Try:
```

```
        # Generate response (with streaming check if supported)
```

```
        Response, metadata = self._generate_with_monitoring(
```

```
            Query, domain, intent, omni_framework
```

```
        )
```

```
        # Track max  $\Delta E$  during reasoning
```

```
        Reasoning_delta_e_max = metadata.get('max_delta_e', 0.0)
```

Break # Success!

Except EthicalDriftException as e:

 Retry_count += 1

 Self._log_violation(

 e.gate_result, e.reasoning_text, domain, intent, blocked=True

)

 Print(f"△ Reasoning aborted (attempt {retry_count}/{max_retries}): {e}")

If retry_count >= max_retries:

 Return {

 'success': False,

 'error': f'Max retries exceeded due to ethical drift',

 'delta_e': e.gate_result.delta_e,

 'response': None

 }

 # Retry with tightened constraints

 Self.ethical_gate.threshold_reasoning *= 0.9

Continued for interested parties

Sulemara@protonmail.com